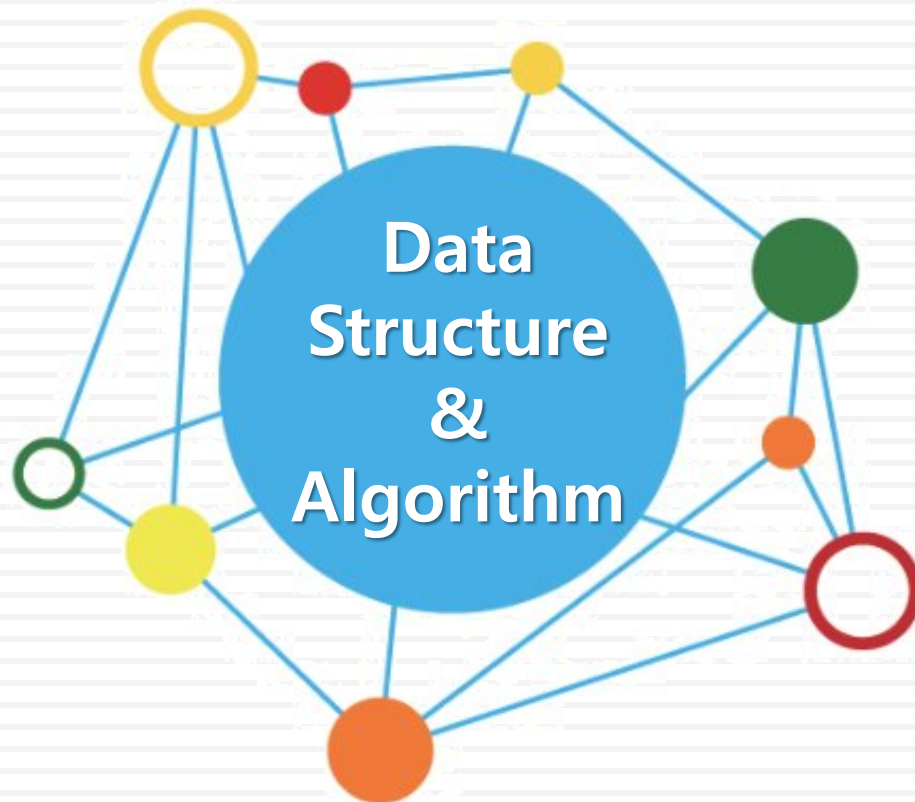


# 데이터 구조론

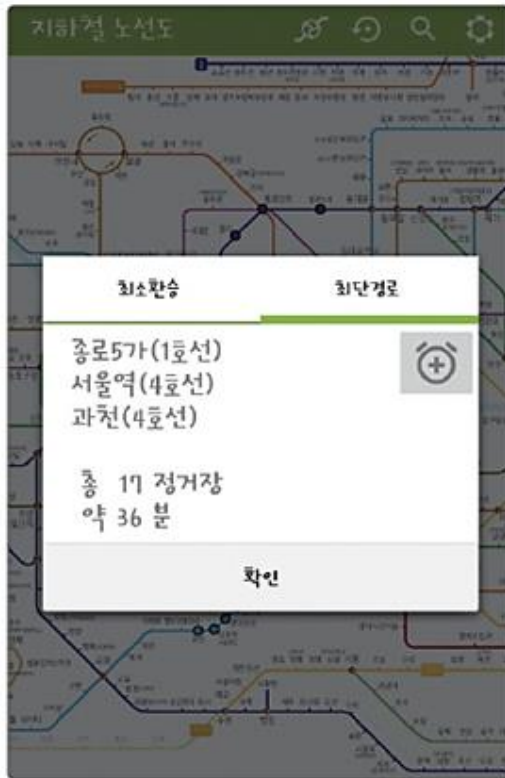


유길현

그래프(Graph)

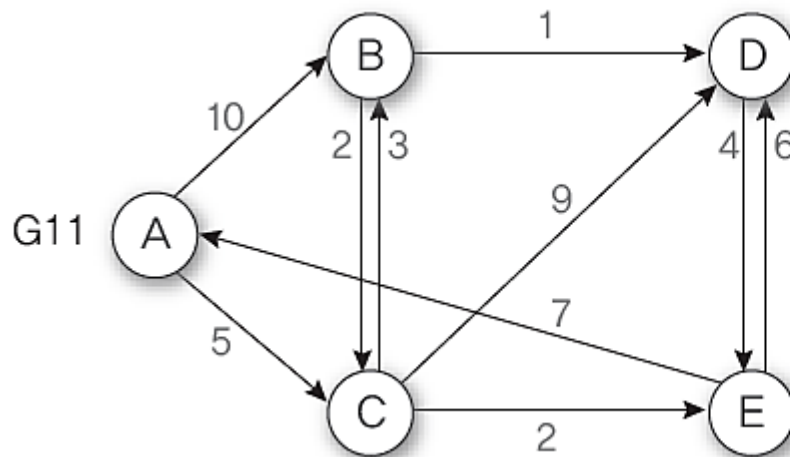
## ❖ 최단 경로(Shortest Path)

- 신장 트리가 아닌 가중치 그래프, 즉 네트워크(Network)에서 정점  $u$ 와 정점  $v$ 를 연결하는 경로 중에서 가중치의 합이 최소인 경로
- 최단 경로 응용 예



## ❖ 최단 경로(Shortest Path)

- 가중치 인접 행렬(Weight Adjacent Matrix)
  - 최단 경로를 구하려는 가중치 그래프의 가중치를 저장
  - 그래프에서 사용한 인접 행렬과 같은 형태의 2차원 배열, 두 정점 사이에 간선이 없으면 0이 아니라  $\infty$ (무한대)값이 저장되어 있다고 가정.
  - 각 정점은 자기 자신과 이어진 간선을 허용하지 않으므로 가중치 인접 행렬에서 대각선 값은 0



|   | A        | B        | C        | D        | E        |
|---|----------|----------|----------|----------|----------|
| A | 0        | 10       | 5        | $\infty$ | $\infty$ |
| B | $\infty$ | 0        | 2        | 1        | $\infty$ |
| C | $\infty$ | 3        | 0        | 9        | 2        |
| D | $\infty$ | $\infty$ | $\infty$ | 0        | 4        |
| E | 7        | $\infty$ | $\infty$ | 6        | 0        |

## ❖ 최단 경로(Shortest path)

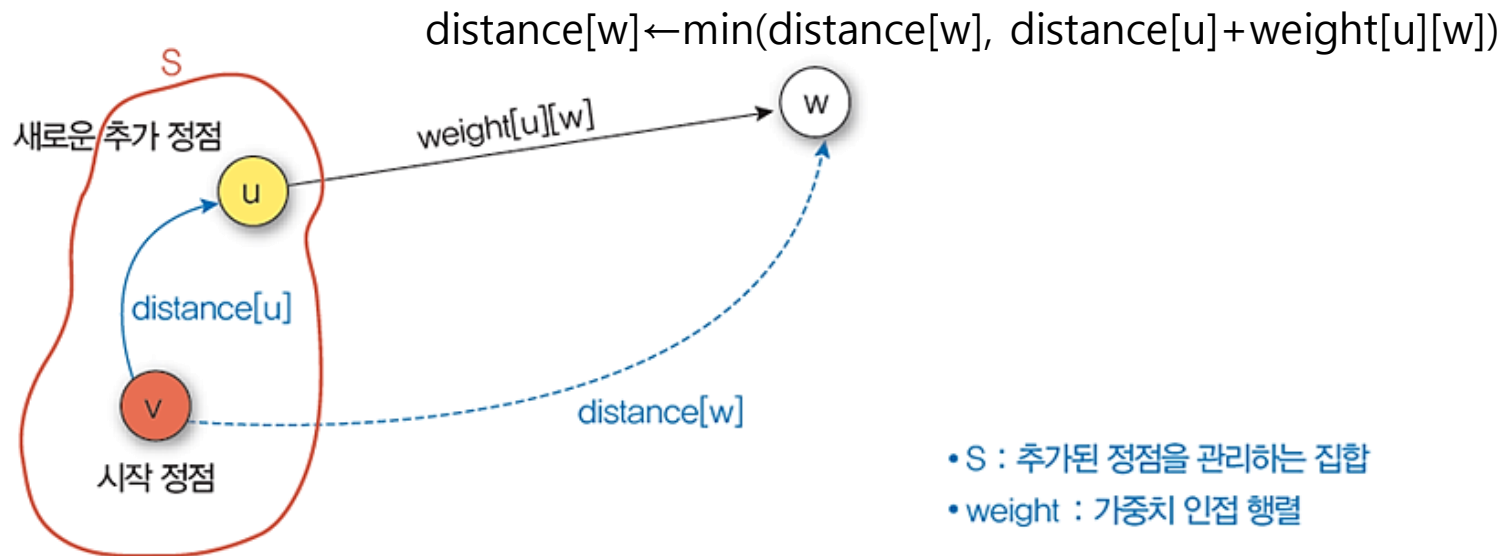
- 다익스트라(Dijkstra)최단 경로 알고리즘
  - 하나의 정점에서 다른 정점까지의 최단 경로를 구함
  - 단일 점에서의 최단 경로(One to All Shortest Path)알고리즘 중 가장 많이 사용
  - 무방향 그래프 방향 그래프에 모두 적용 가능
  - 알고리즘에서 최단 경로 찾기 순서

- ① 경로의 길이를 저장할 배열 distance 준비 : 시작 정점으로부터 각 정점에 이르는 경로의 길이를 저장하기 위한 배열 distance를  $\infty$ (무한대)로 초기화 한다.
- ② 시작 정점 초기화 : 시작 정점의 distance를 0으로 초기화하고 집합 S에 추가 한다.
- ③ 최단 거리 수정 : 집합 S에 속하지 않은 정점 중에서 distance가 최소인 정점 u를 찾아 집합 S에 추가한다. 새로운 정점 u가 추가되면 u에 인접하고 집합 S에 포함되지 않은 정점 w의 distance값을 다음 식에 따라 수정한다. 집합 S에 모든 정점이 추가될 때까지 ③을 반복한다.

$$\text{distance}[w] \leftarrow \min(\text{distance}[w], \text{distance}[u] + \text{weight}[u][w])$$

## ❖ 다익스트라(Dijkstra)최단 경로 알고리즘

- 시작 정점  $v$ 에서 정점  $u$ 까지의 최단 경로  $distance[u]$ 를 구해 정점  $u$ 가 집합  $S$ 에 추가되면 새로 추가된 정점  $u$ 에 의해 단축되는 경로가 있는지를 확인함. 즉, 현재 알고 있는  $distance[w]$ 를 새로 추가된 정점  $u$ 를 거쳐서 가는  $distance[u] + weight[u][w]$ 를 계산한 경로와 비교해 경로가 단축되면  $distance[w]$ 를 단축된 경로 값으로 수정함으로써 최단 경로를 찾음
- 최단 경로 구하기 기본 개념



## ❖ 다익스트라(Dijkstra)최단 경로 알고리즘

### 알고리즘

### 다익스트라 최단 경로

**Dijkstra\_shortestPath**(G, v)

$S \leftarrow \{v\};$

for( $i \leftarrow 0; i \in V(G); i \leftarrow i+1$ ) do

$distance[i] \leftarrow weight[v][i];$

for( $i \leftarrow 0; S \neq V(G); i \leftarrow i+1$ ) do { // 집합 S와 정점의  $V(G)$ 가 같지 않은 동안

$u \leftarrow S$ 에 속하지 않은 정점 중에서  $distance[]$ 가 최소인 정점;

$S \leftarrow S \cup \{u\};$

    for( $w \leftarrow 0; w \in V(G); w \leftarrow w+1$ ) do

        if(  $z \notin S$  ) then

$distance[w] \leftarrow \min(distance[w], distance[u] + weight[u][w]);$

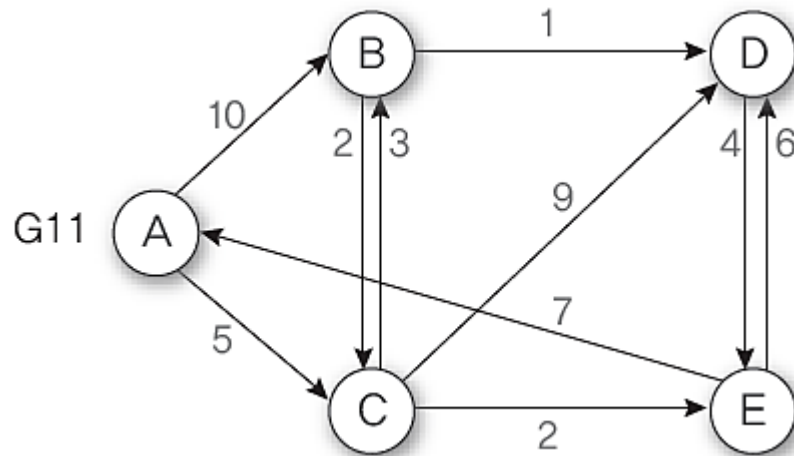
}

**End Dijkstra\_shortestPath()**

## ❖ 다익스트라(Dijkstra)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

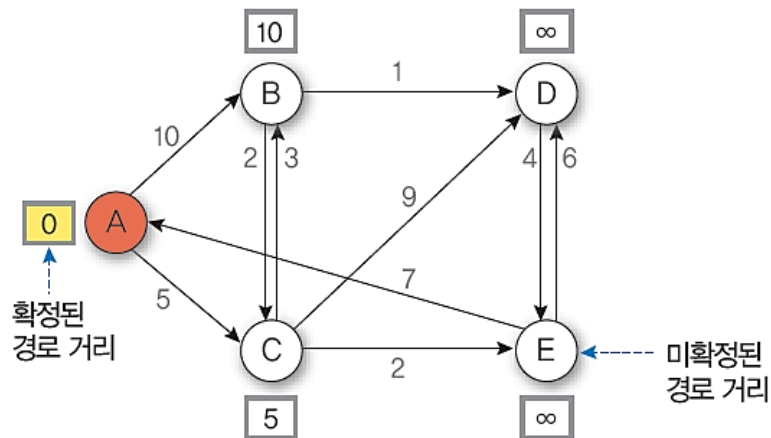
- 시작 정점은 A, 각 정점 옆에 붙은 숫자는 시작 정점 A에서 각 정점까지 도달하는 거리를 의미. 집합 S에 포함되는 추가 경로 거리는 노란색, 집합 S의 정점은 빨간색, 현재 단계에서 새로 추가된 정점과 그로 인해 수정된 distance값은 파란색으로 표시



## ❖ 다익스트라(Dijkstra)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

- 초기 상태 : 시작 정점 A를 집합 S의 초기값으로 설정, 시작 정점 A에서의 인접 정점에 대한 가중치 값을 인접 행렬 weight에서 구해 배열 distance의 초기값으로 설정



|   | A | B  | C | D | E |
|---|---|----|---|---|---|
| A | 0 | 10 | 5 | ∞ | ∞ |
| B | ∞ | 0  | 2 | 1 | ∞ |
| C | ∞ | 3  | 0 | 9 | 2 |
| D | ∞ | ∞  | ∞ | 0 | 4 |
| E | 7 | ∞  | ∞ | 6 | 0 |

$S = \{A\}$

|          | A | B  | C | D | E |
|----------|---|----|---|---|---|
| distance | 0 | 10 | 5 | ∞ | ∞ |

집합 S에 포함되지 않은 정점 중 최솟값



## ❖ 다익스트라(Dijkstra)최단 경로 알고리즘

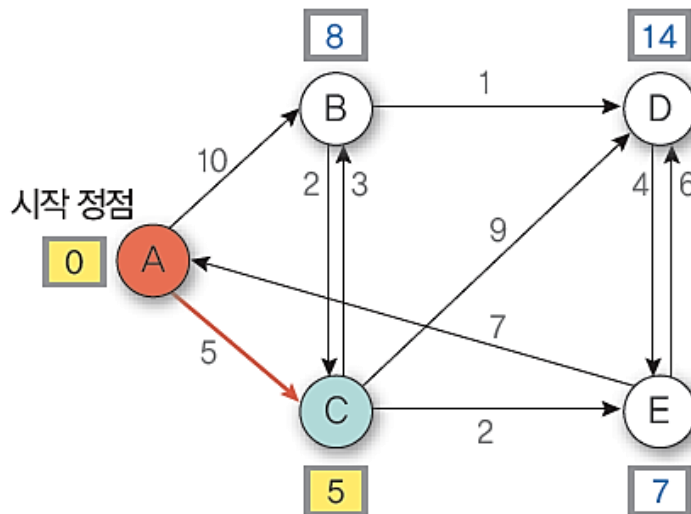
### ■ 최단 경로를 구하는 과정

- ① 시작 정점 A의 진출 간선과 연결된 인접 정점 B와 C 중에서 최단 경로인 C를 집합 S에 추가. 새로 추가된 정점 C에 의해 단축되는 경로가 있는지 확인하여 배열 distance값을 수정

$$\text{distance}[B] \leftarrow \min(\text{distance}[B], \text{distance}[C] + \text{weight}[C][B]) = \min(10, 5+3) = 8$$

$$\text{distance}[D] \leftarrow \min(\text{distance}[D], \text{distance}[C] + \text{weight}[C][D]) = \min(\infty, 5+9) = 14$$

$$\text{distance}[E] \leftarrow \min(\text{distance}[E], \text{distance}[C] + \text{weight}[C][E]) = \min(\infty, 5+2) = 7$$



$S = \{A, C\}$

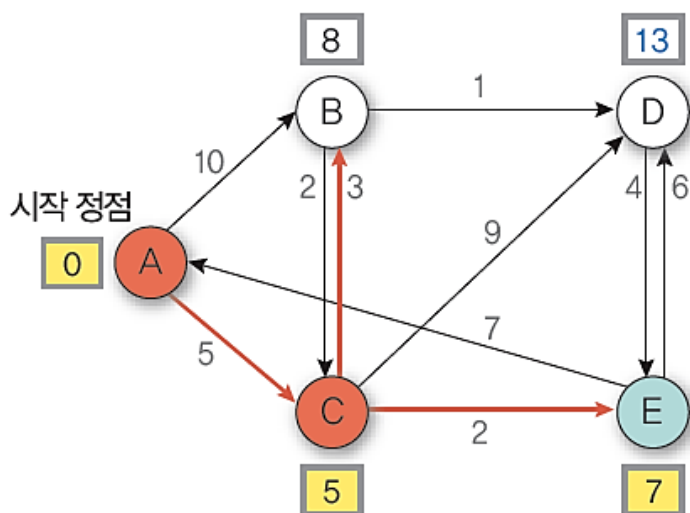
|          | A | B | C | D  | E |
|----------|---|---|---|----|---|
| distance | 0 | 8 | 5 | 14 | 7 |

## ❖ 다익스트라(Dijkstra)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

- ② 집합 S에 포함되지 않은 정점 중에서 최단 경로인 E를 집합 S에 추가. 새로 추가된 정점 E에 의해 단축되는 경로가 있는지 확인하여 배열 distance값을 수정

$$\text{distance}[D] \leftarrow \min(\text{distance}[D], \text{distance}[E] + \text{weight}[E][D]) = \min(14, 7+6) = 13$$



$$S = \{A, C, E\}$$

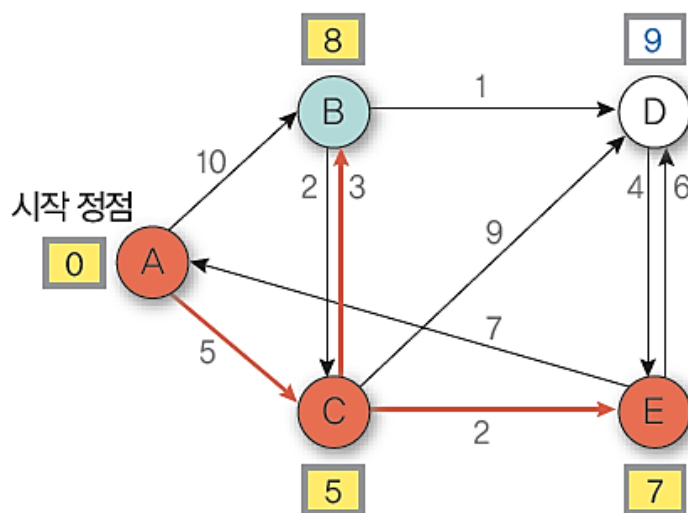
|          | A | B | C | D  | E |
|----------|---|---|---|----|---|
| distance | 0 | 8 | 5 | 13 | 7 |

## ❖ 다익스트라(Dijkstra)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

- ③ 집합 S에 포함되지 않은 정점 중에서 최단 경로인 B를 집합 S에 추가. 새로 추가된 정점 B에 의해 단축되는 경로가 있는지 확인하여 배열 distance값을 수정

$$\text{distance}[D] \leftarrow \min(\text{distance}[D], \text{distance}[B] + \text{weight}[B][D]) = \min(13, 8+1) = 9$$



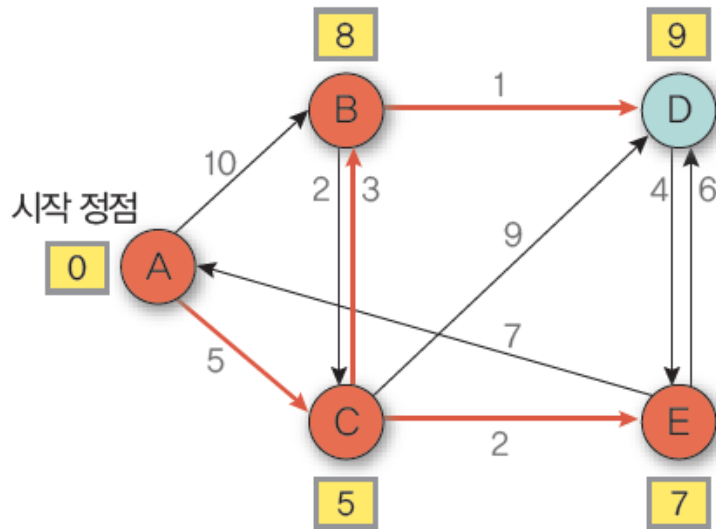
$$S = \{A, B, C, E\}$$

|          | A | B | C | D | E |
|----------|---|---|---|---|---|
| distance | 0 | 8 | 5 | 9 | 7 |

## ❖ 다익스트라(Dijkstra)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

- ④ 집합 S에 포함되지 않은 정점 중에서 최단 경로인 D를 집합 S에 추가. 이로써 모든 정점이 집합 S에 추가되었으므로 최단 경로가 완성됨

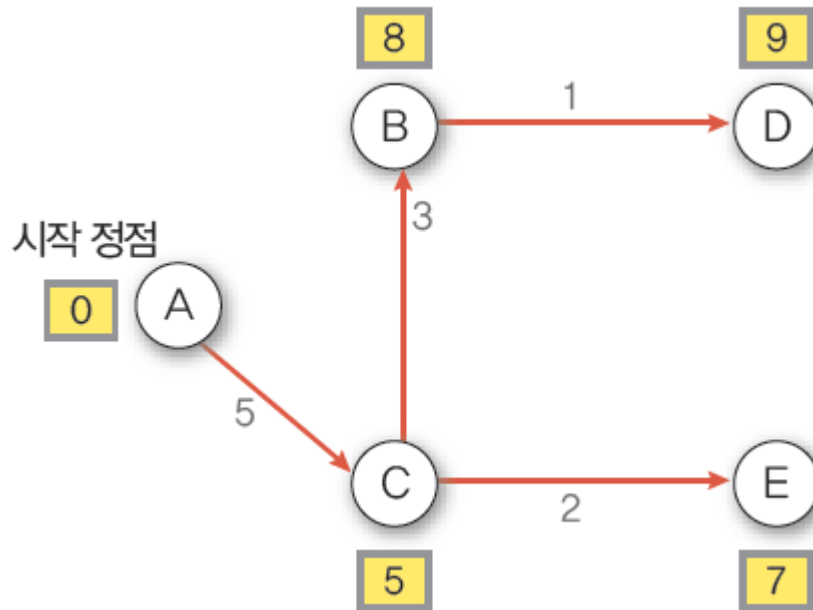


$S = \{A, B, C, D, E\}$

|          | A | B | C | D | E |
|----------|---|---|---|---|---|
| distance | 0 | 8 | 5 | 9 | 7 |

## ❖ 다익스트라(Dijkstra)최단 경로 알고리즘

- 최단 경로를 구하는 과정
  - 그래프 G11의 최단 경로 트리



## ❖ 다익스트라(Dijkstra)최단 경로 구하기 프로그램

```
#include "stdio.h"
#include "limits.h"

#define TRUE 1
#define FALSE 0
#define MAX_VERTICES 5// 그래프의 정점 개수
#define INF 10000

// 그래프 G11의 가중치 인접행렬
int weight[MAX_VERTICES][MAX_VERTICES] = {
    { 0, 10, 5, INF, INF },
    { INF, 0, 2, 1, INF },
    { INF, 3, 0, 9, 2 },
    { INF, INF, INF, 0, 4 },
    { 7, INF, INF, 6, 0 },
};

int distance[MAX_VERTICES]; // 시작 정점으로부터의 최단 경로 길이 저장
int S[MAX_VERTICES];       // 정점의 집합 S
```

## ❖ 다익스트라(Dijkstra)최단 경로 구하기 프로그램

```
// 최소 거리를 갖는 다음 정점을 찾는 연산
int nextVertex(int n)
{
    int i, min, minPos;
    min = INT_MAX;
    minPos = -1;
    for (i = 0; i < n; i++)
        if ((distance[i] < min) && !S[i])
        {
            min = distance[i];
            minPos = i;
        }
    return minPos;
}
```

## ❖ 다익스트라(Dijkstra)최단 경로 구하기 프로그램

```
// 최단 경로 구하는 과정을 출력하는 연산
int printStep(int step)
{
    int i;
    printf("\n %3d 단계 : S={", step);
    for (i = 0; i < MAX_VERTICES; i++)
        if (S[i] == TRUE) printf("%3c", i + 65);

    if (step < 1) printf(" } \n");
    else if (step < 4) printf(" } \n");
    else printf(" } \n");

    printf(" distance :[ ");
    for (i = 0; i < MAX_VERTICES; i++)
        if (distance[i] == 10000)
            printf("%4c", '*');
        else
            printf("%4d", distance[i]);
    printf("%4c", ']');
    return ++step;
}
```



## ❖ 다익스트라(Dijkstra)최단 경로 구하기 프로그램

```
void Dijkstra_shortestPath(int start, int n)
{
    int i, u, w, step = 0;

    for (i = 1; i < n; i++) { // 초기화
        distance[i] = weight[start][i];
        S[i] = FALSE;
    }

    S[start] = TRUE; // 시작 정점을 집합 S에 추가
    distance[start] = 0; // 시작 정점의 최단경로를 0으로 설정

    step = printStep(0); // 0단계 상태를 출력
```

## ❖ 다익스트라(Dijkstra)최단 경로 구하기 프로그램

```
for (i = 0; i < n - 1; i++)
{
    u = nextVertex(n);      // 최단 경로를 만드는 다음 정점 u 찾기
    S[u] = TRUE;            // 정점 u를 집합 S에 추가
    for (w = 0; w < n; w++)
        if (!S[w])          // 집합 S에 포함되지 않은 정점 중에서
            if (distance[u] + weight[u][w] < distance[w])
                distance[w] = distance[u] + weight[u][w]; // 경로 길이 수정

    step = printStep(step); // 현재 단계 출력
}
}
```

## ❖ 다익스트라(Dijkstra)최단 경로 구하기 프로그램

```
void main(void)
{
    int i, j;
    printf("\n ***** 가중치 인접 행렬 *****\n\n");
    for (i = 0; i < MAX_VERTICES; i++)
    {
        for (j = 0; j < MAX_VERTICES; j++)
        {
            if (weight[i][j] == 10000)
                printf("%4c", '*');
            else
                printf("%4d", weight[i][j]);
        }
        printf("\n\n");
    }

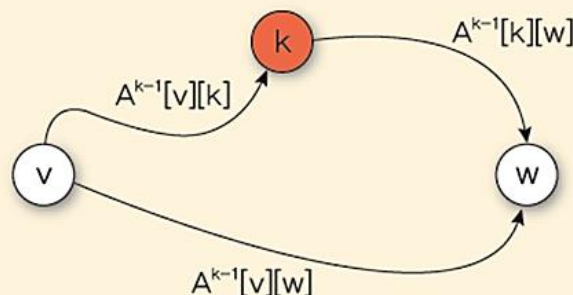
    printf("\n ***** 다익스트라 최단 경로 구하기 *****\n\n");
    Dijkstra_shortestPath(0, MAX_VERTICES);

    getchar();
}
```

## ❖ 플로이드(Floyd)최단 경로 알고리즘

- 모든 정점 사이의 최단 경로(All to All shortest Path)를 구한 것
- k-최단 경로 – 플로이드 최단 경로 알고리즘으로 만든 최단 경로

- $A^k[v][w]$ 는 0부터 k까지의 정점만을 이용한 정점 v에서 정점 w까지의 최단 경로를 의미한다.
- 정점 0부터 정점 k-1까지의 정점을 고려한 최단 거리  $A^{k-1}$ 을 구한 상태에서 다음 정점 k를 고려할 때,  $A^{k-1}[v][w]$ 와  $A^{k-1}[v][k]+A^{k-1}[k][w]$  중에서 작은 값에 따라 최단 경로가 수정된다.



$$A^k[v][w] \leftarrow \min(A^{k-1}[v][w], A^{k-1}[v][k] + A^{k-1}[k][w])$$

- $A^{-1} \rightarrow A^0 \rightarrow A^1 \rightarrow A^2 \rightarrow \dots$  순서로 경로에 정점을 늘려 가면서 최단 경로를 구하다가 최종적으로  $A^{n-1}$ 을 구하면 n개의 모든 정점 사이의 최단 경로를 구하게 된다.
- $A^{-1}$ 은 초기 상태로서 가중치 인접 행렬인 배열 weight값이 되고,  $A^{n-1}[v][w]$ 는 정점 0부터 정점 n-1까지의 모든 정점을 이용한 최단 경로가 된다.

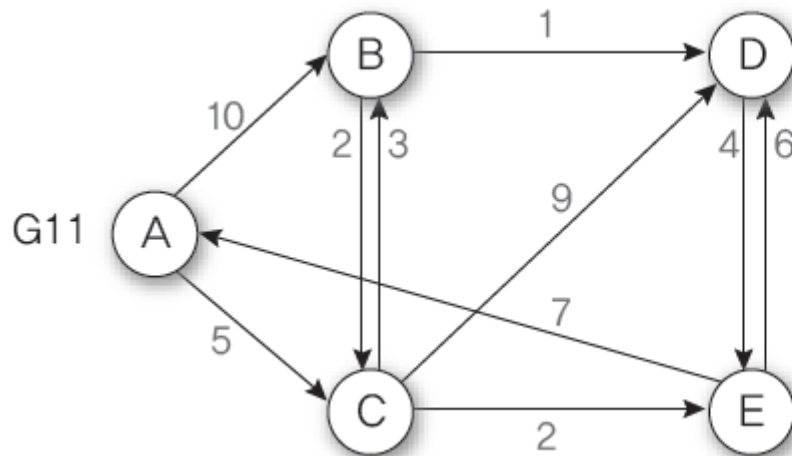
## ❖ 플로이드(Floyd)최단 경로 알고리즘

| 알고리즘                                                                                                                                                                                                                                   | 플로이드 최단 경로 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <pre><b>Floyd_shortestPath</b>(G, v)   for(k ← 0; k ≤ n-1; k ← k+1) do     for(v ← 0; v ≤ n-1; v ← v+1) do       for(w ← 0; w ≤ n-1; w ← w+1) do         A[v][w] ← min(A[v][w], a[v][k]+A[k][w]);       End Floyd_shortestPath()</pre> |            |

## ❖ 플로이드(Floyd)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

- 그래프 G11의 정점 A, B, C, D, E는 k의 0, 1, 2, 3, 4에 대응됨



|   | A        | B        | C        | D        | E        |
|---|----------|----------|----------|----------|----------|
| A | 0        | 10       | 5        | $\infty$ | $\infty$ |
| B | $\infty$ | 0        | 2        | 1        | $\infty$ |
| C | $\infty$ | 3        | 0        | 9        | 2        |
| D | $\infty$ | $\infty$ | $\infty$ | 0        | 4        |
| E | 7        | $\infty$ | $\infty$ | 6        | 0        |

## ❖ 플로이드(Floyd)최단 경로 알고리즘

- 최단 경로를 구하는 과정

- 초기 상태  $A^{-1}$  : 인접 행렬 weight를 배열 A에 복사하여 초기화

| $A^{-1}$ | A        | B        | C        | D        | E        |
|----------|----------|----------|----------|----------|----------|
| A        | 0        | 10       | 5        | $\infty$ | $\infty$ |
| B        | $\infty$ | 0        | 2        | 1        | $\infty$ |
| C        | $\infty$ | 3        | 0        | 9        | 2        |
| D        | $\infty$ | $\infty$ | $\infty$ | 0        | 4        |
| E        | 7        | $\infty$ | $\infty$ | 6        | 0        |

## ❖ 플로이드(Floyd)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

- ①  $A^0$  : 두 정점 사이의 최단 경로에서 정점 A를 거쳐서 가는 경로를 고려하여 최단 경로를 수정

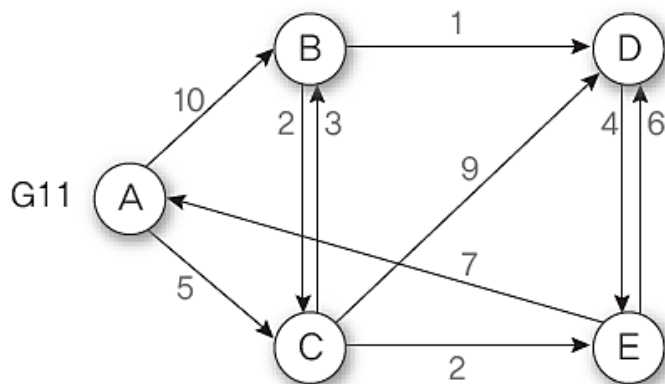
| $A^0$ | A        | B        | C        | D        | E        |
|-------|----------|----------|----------|----------|----------|
| A     | 0        | 10       | 5        | $\infty$ | $\infty$ |
| B     | $\infty$ | 0        | 2        | 1        | $\infty$ |
| C     | $\infty$ | 3        | 0        | 9        | 2        |
| D     | $\infty$ | $\infty$ | $\infty$ | 0        | 4        |
| E     | 7        | 17       | 12       | 6        | 0        |

$$A^0[E][B] \leftarrow \min(A^{-1}[E][B], A^{-1}[E][A] + A^{-1}[A][B])$$

$$= \min(\infty, 7 + 10) = 17$$

$$A^0[E][C] \leftarrow \min(A^{-1}[E][C], A^{-1}[E][A] + A^{-1}[A][C])$$

$$= \min(\infty, 7 + 5) = 12$$





## ❖ 플로이드(Floyd)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

②  $A^1$  :  $A^0$ 에서 구한 경로와 정점 B를 추가로 거쳐서 가는 경로를 고려하여 최단 경로를 수정

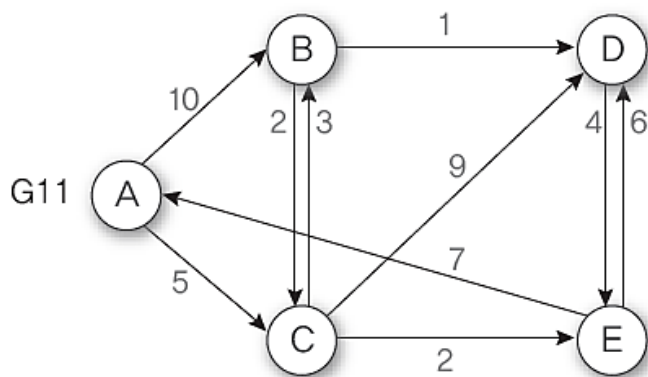
| $A^1$ | A        | B        | C        | D  | E        |
|-------|----------|----------|----------|----|----------|
| A     | 0        | 10       | 5        | 11 | $\infty$ |
| B     | $\infty$ | 0        | 2        | 1  | $\infty$ |
| C     | $\infty$ | 3        | 0        | 4  | 2        |
| D     | $\infty$ | $\infty$ | $\infty$ | 0  | 4        |
| E     | 7        | 17       | 12       | 6  | 0        |

$$A^1[A][D] \leftarrow \min(A^0[A][D], A^0[A][B] + A^0[B][D])$$

$$= \min(\infty, 10 + 1) = 11$$

$$A^1[C][D] \leftarrow \min(A^0[C][D], A^0[C][B] + A^0[B][D])$$

$$= \min(9, 3 + 1) = 4$$

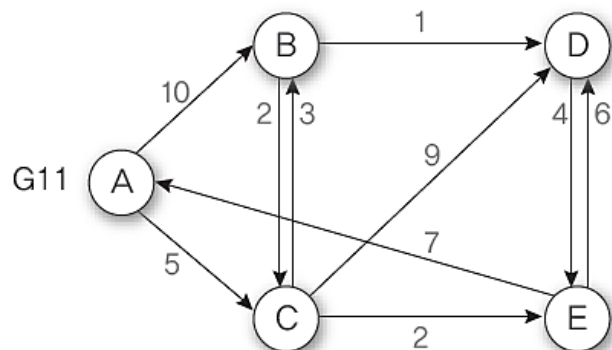


## ❖ 플로이드(Floyd)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

③  $A^2$  :  $A^1$ 에서 구한 경로와 정점 C를 추가로 거쳐서 가는 경로를 고려하여 최단 경로를 수정

| $A^2$ | A        | B        | C        | D | E |
|-------|----------|----------|----------|---|---|
| A     | 0        | 8        | 5        | 9 | 7 |
| B     | $\infty$ | 0        | 2        | 1 | 4 |
| C     | $\infty$ | 3        | 0        | 4 | 2 |
| D     | $\infty$ | $\infty$ | $\infty$ | 0 | 4 |
| E     | 7        | 15       | 12       | 6 | 0 |



$$A^2[A][B] \leftarrow \min(A^1[A][B], A^1[A][C] + A^1[C][B])$$

$$= \min(10, 5+3) = 8$$

$$A^2[A][D] \leftarrow \min(A^1[A][D], A^1[A][C] + A^1[C][D])$$

$$= \min(11, 5+4) = 9$$

$$A^2[A][E] \leftarrow \min(A^1[A][E], A^1[A][C] + A^1[C][E])$$

$$= \min(\infty, 5+2) = 7$$

$$A^2[B][E] \leftarrow \min(A^1[B][E], A^1[B][C] + A^1[C][E])$$

$$= \min(\infty, 2+2) = 4$$

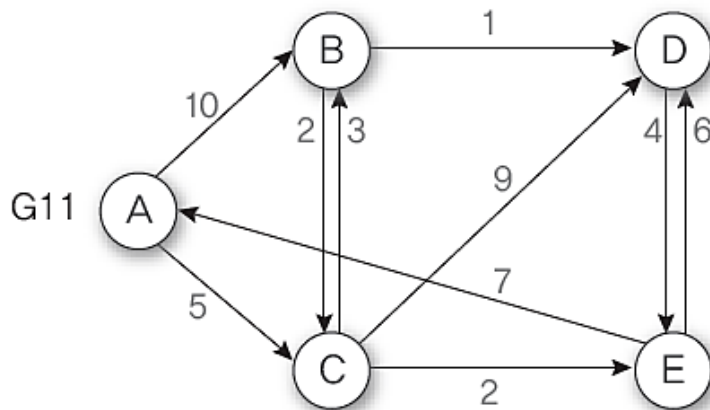
$$A^2[E][B] \leftarrow \min(A^1[E][B], A^1[E][C] + A^1[C][B])$$

$$= \min(17, 12+3) = 15$$

## ❖ 플로이드(Floyd)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

- ④  $A^3$  :  $A^2$ 에서 구한 경로와 정점 D를 추가로 거쳐서 가는 경로를 고려하여 최단 경로를 수정 정점 D를 추가로 거쳐가는 경우에 단축되는 경로가 없어  $A^3=A^2$ 가 됨



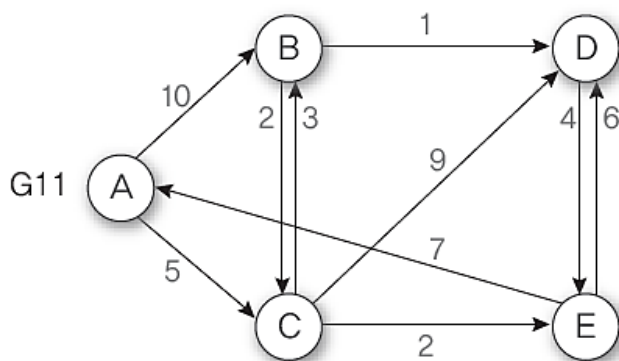
| $A^3$ | A        | B        | C        | D | E |
|-------|----------|----------|----------|---|---|
| A     | 0        | 8        | 5        | 9 | 7 |
| B     | $\infty$ | 0        | 2        | 1 | 4 |
| C     | $\infty$ | 3        | 0        | 4 | 2 |
| D     | $\infty$ | $\infty$ | $\infty$ | 0 | 4 |
| E     | 7        | 15       | 12       | 6 | 0 |

## ❖ 플로이드(Floyd)최단 경로 알고리즘

### ■ 최단 경로를 구하는 과정

⑤  $A^4$  :  $A^3$ 에서 구한 경로와 정점 E를 추가로 거쳐서 가는 경로를 고려하여 최단 경로를 수정

| $A^4$ | A  | B  | C  | D | E |
|-------|----|----|----|---|---|
| A     | 0  | 8  | 5  | 9 | 7 |
| B     | 11 | 0  | 2  | 1 | 4 |
| C     | 9  | 3  | 0  | 4 | 2 |
| D     | 11 | 19 | 16 | 0 | 4 |
| E     | 7  | 15 | 12 | 6 | 0 |



$$A^4[B][A] \leftarrow \min(A^3[B][A], A^3[B][E] + A^3[E][A])$$

$$= \min(\infty, 4 + 7) = 11$$

$$A^4[C][A] \leftarrow \min(A^3[C][A], A^3[C][E] + A^3[E][A])$$

$$= \min(\infty, 2 + 7) = 9$$

$$A^4[D][A] \leftarrow \min(A^3[D][A], A^3[D][E] + A^3[E][A])$$

$$= \min(\infty, 4 + 7) = 11$$

$$A^4[D][B] \leftarrow \min(A^3[D][B], A^3[D][E] + A^3[E][B])$$

$$= \min(\infty, 4 + 15) = 19$$

$$A^4[D][C] \leftarrow \min(A^3[D][C], A^3[D][E] + A^3[E][C])$$

$$= \min(\infty, 4 + 12) = 16$$

## ❖ 플로이드(Floyd)최단 경로 구하기 프로그램

```
#include "stdio.h"
#include "limits.h"

#define MAX_VERTICES 5// 그래프의 정점 개수
#define INF 10000

// 그래프 G11의 가중치 인접행렬
int weight[MAX_VERTICES][MAX_VERTICES] = {
    { 0,10,5,INF,INF },
    { INF,0,2,1,INF },
    { INF, 3,0,9,2 },
    { INF,INF,INF,0,4 },
    { 7,INF,INF,6,0 },
};

int A[MAX_VERTICES][MAX_VERTICES];    // k-최단 경로 배열
```

## ❖ 플로이드(Floyd)최단 경로 구하기 프로그램

```
// 최단 경로를 구하는 과정을 출력하는 연산
void printStep(int step)
{
    int i, j;
    printf("\n A%d : ", step);
    for (i = 0; i < MAX_VERTICES; i++)
    {
        printf("\t");
        for (j = 0; j < MAX_VERTICES; j++)
        {
            if (A[i][j] == 10000)
                printf("%4c", '*');
            else
                printf("%4d", A[i][j]);
        }
        printf("\n\n");
    }
}
```

## ❖ 플로이드(Floyd)최단 경로 구하기 프로그램

```
void Floyd_shortestPath(int n)
{
    int v, w, k, step = -1;

    for (v = 0; v < n; v++) // 초기화
        for (w = 0; w < n; w++)
            A[v][w] = weight[v][w];

    printStep(step);

    for (k = 0; k < n; k++)
    {
        for (v = 0; v < n; v++)
            for (w = 0; w < n; w++)
                if (A[v][k] + A[k][w] < A[v][w])
                    A[v][w] = A[v][k] + A[k][w];
        printStep(++step);
    }
}
```

## ❖ 플로이드(Floyd)최단 경로 구하기 프로그램

```
void main(void)
{
    int i, j;
    printf("\n ***** 가중치 인접 행렬 *****\n\n");
    for (i = 0; i < MAX_VERTICES; i++)
    {
        for (j = 0; j < MAX_VERTICES; j++)
        {
            if (weight[i][j] == 10000)
                printf("%4c", '*');
            else
                printf("%4d", weight[i][j]);
        }
        printf("\n\n");
    }

    printf("\n ***** 플로이드 최단 경로 구하기 *****\n\n");
    Floyd_shortestPath(MAX_VERTICES);

    getchar();
}
```



## ❖ 검색(search)

- 컴퓨터에 저장한 자료 중에서 원하는 항목을 찾는 작업
  - 검색 성공 - 원하는 항목을 찾은 경우
  - 검색 실패 - 원하는 항목을 찾지 못한 경우
- 탐색 키를 가진 항목을 찾는 것
  - 탐색 키search key - 자료를 구별하여 인식할 수 있는 키
- 삽입/삭제 작업에서의 검색
  - 원소를 삽입하거나 삭제할 위치를 찾기 위해서 검색 연산 수행

## ❖ 검색 방법

- 수행 위치에 따른 분류
  - 내부 검색(Internal Search) : 메모리 내의 자료에 대해서 검색 수행
  - 외부 검색(External Search) : 보조 기억 장치에 있는 자료에 대해서 검색 수행
- 검색 방식에 따른 분류
  - 비교 검색 방식(comparison search method)
  - 계산 검색 방식(non-comparison method)

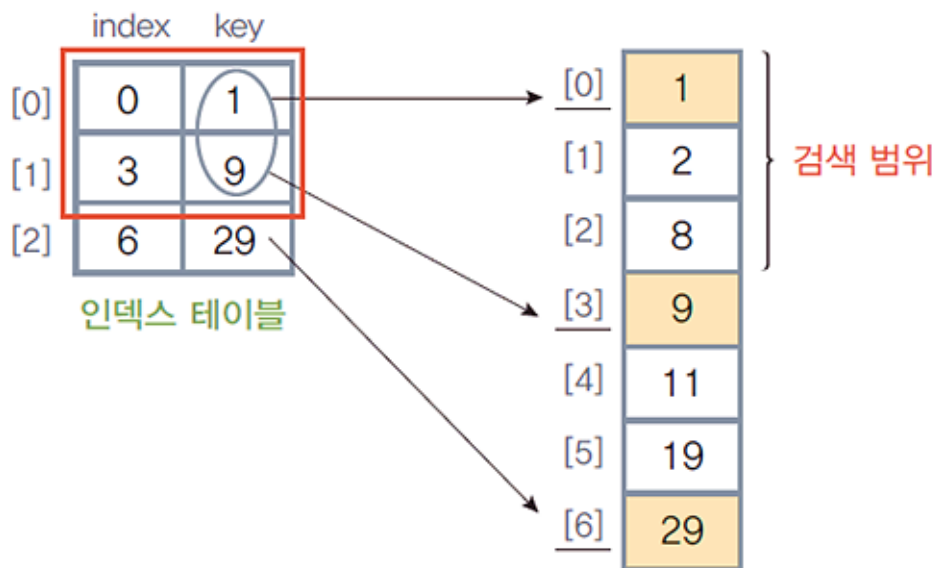
| 검색 방법 | 설명                      | 종류                    |
|-------|-------------------------|-----------------------|
| 비교 검색 | 검색 대상의 키를 비교하여 검색한다.    | 순차 검색, 이진 검색<br>트리 검색 |
| 계산 검색 | 계수적인 성질을 이용한 계산으로 검색한다. | 해싱                    |

## ❖ 색인 순차 검색(index sequential search)

- 정렬되어 있는 자료에 대한 인덱스 테이블(index table)을 추가로 사용하여 탐색 효율을 높인 검색 방법
- 인덱스 테이블
  - 배열에 정렬되어 있는 자료 중에서 일정한 간격으로 떨어져 있는 원소들을 저장한 테이블
    - 자료가 저장되어 있는 배열의 크기가  $n$ 이고 인덱스 테이블의 크기가  $m$ 일 때, 배열에서  $n/m$ 간격으로 떨어져 있는 원소와 그의 인덱스를 인덱스 테이블에 저장
- 검색 방법
  - $\text{indexTable}[i].\text{key} \leq \text{key} < \text{indexTable}[i+1].\text{key}$ 를 만족하는  $i$ 를 찾아서 배열의 어느 범위에 있는지를 먼저 알아낸 후에 해당 범위에 대해서만 순차 검색 수행

## ❖ 색인 순차 검색

- 색인 순차 검색 (예) 검색 대상 자료 : {1, 2, 8, 9, 11, 19, 29}
  - 키 값이 6인 원소를 검색하려면 먼저 인덱스 테이블의 키 값을 검색.
    - 키 값 6은  $\text{indexTable}[0].\text{key} \leq 6 < \text{indexTable}[1].\text{key}$ 로 인덱스 테이블의 키값 1과 9 사이에 있음.
    - 검색 범위는  $\text{indexTable}[0].\text{index} \leq \text{검색 범위} < \text{indexTable}[1].\text{index}$ 가 되므로 자료가 저장되어 있는 배열의 인덱스 0번부터 2번 까지를 검색 범위로 정하고 순차 검색을 수행



## ❖ 색인 순차 검색을 위해 인덱스 테이블 알고리즘

| 알고리즘                                                                                                                                                                                                                                                                                                                              | 인덱스 테이블 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <pre><b>makeIndexTable(a[ ], n, key)</b><br/>  n ← size/m;           // 인덱스 테이블에 들어가는 배열 원소의 간격 계산<br/>  if (size mod m &gt; 0) then n ← n + 1;<br/>  for(i ← 0; i &lt; m; i ← i + 1) do {    // 인덱스 테이블 채우기<br/>    indexTable[i].index ← i*n;<br/>    indexTable[i].key ← a[i*n];<br/>  }<br/><b>end makeIndexTable()</b></pre> |         |

## ❖ 색인 순차 검색 알고리즘

### 알고리즘

### 정렬된 자료의 색인 순차 검색

```
indexSearch(a[ ], n, key)
  for (i ← 0; i < m; i ← i+1) do
    if ((indexTable[i].key ≤ key) and (indexTable[i + 1].key > key)) then {
      begin ← indexTable[i].index;
      end ← indexTable[i + 1].index;
      break;
    }
  sequentialSearch2(a[], begin, end, key);
end indexSearch()
```

## ❖ 자료 색인 순차 검색하기 프로그램

```
include <stdio.h>
#define INDEX_SIZE 3                // 인덱스 테이블의 크기를 3으로 정의
typedef int element;

// 인덱스 테이블의 구조(index, key)를 정의
typedef struct {
    int index;
    element key;
} iTable;

iTable indexTable[INDEX_SIZE];      // 인덱스 테이블 indexTable 생성
```

## ❖ 자료 색인 순차 검색하기 프로그램

```
// 배열 a의 begin 원소와 end 원소 사이에서 key를 순차 검색: [알고리즘 10-2]
void sequentialSearch2(element a[], int begin, int end, element key)
{
    int i = begin;

    printf("\n %d를 검색하라! -> > ", key);
    while (i < end && a[i] < key)
        i++;

    if (a[i] == key)
        printf("%d번째에 검색 성공!\n\n", (i - begin) + 1);
    else
        printf("%d번째에 검색 실패! \n\n", (i - begin) + 1);
}
```



## ❖ 자료 색인 순차 검색하기 프로그램

```
// 배열 a에 대한 인덱스 테이블 만들기
void makeIndexTable(element a[], int size)
{
    int i, n;
    n = size / INDEX_SIZE;  // 인덱스 테이블에 들어가는 배열 원소의 간격 계산
    if (size % INDEX_SIZE > 0)
        n = n + 1;
    for (i = 0; i < INDEX_SIZE; i++) {    // 인덱스 테이블의 채우기
        indexTable[i].index = i*n;
        indexTable[i].key = a[i*n];
    }
}
```

## ❖ 자료 색인 순차 검색하기 프로그램

```
// 색인 순차 검색
void indexSearch(element a[], int n, element key)
{
    int i, begin, end;
    if (key < a[0] || key > a[n - 1])
        printf("₩n찾는 키가 없습니다. 검색 실패! ₩n");

    // 인덱스 테이블을 검색하여 검색 범위 결정
    for (i = 0; i < INDEX_SIZE; i++)
        if ((indexTable[i].key <= key) && (indexTable[i + 1].key > key)) {
            begin = indexTable[i].index;
            end = indexTable[i + 1].index;
            break;
        }
    if (i == INDEX_SIZE) {
        begin = indexTable[i - 1].index;
        end = n;
    }
    sequentialSearch2(a, begin, end, key);    // 검색 범위에 대한 순차검색 수행
}
```

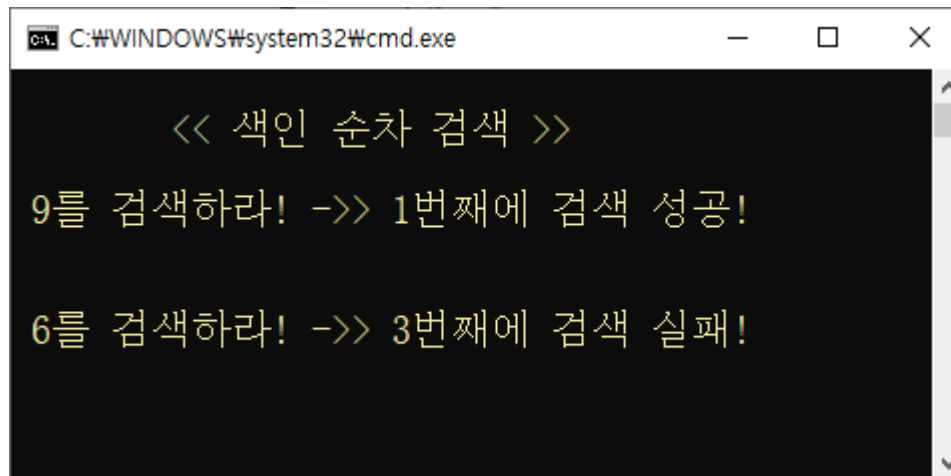
## ❖ 자료 색인 순차 검색하기 프로그램

```
void main()
{
    element a[] = { 1, 2, 8, 9, 11, 19, 29 };
    int n = 7;
    printf("\n\n\t<< 색인 순차 검색 >>\n\n");
    makeIndexTable(a, n);
    indexSearch(a, n, 9); // 배열 a의 n개 원소 중에서 탐색키가 9인 원소 검색
    indexSearch(a, n, 6); // 배열 a의 n개 원소 중에서 탐색키가 6인 원소 검색

    getchar();
}
```

❖ 자료 색인 순차 검색하기 프로그램 : [교재 564p](#)

■ 실행 결과



```
C:\WINDOWS\system32\cmd.exe

<< 색인 순차 검색 >>

9를 검색하라! ->> 1번째에 검색 성공!

6를 검색하라! ->> 3번째에 검색 실패!
```

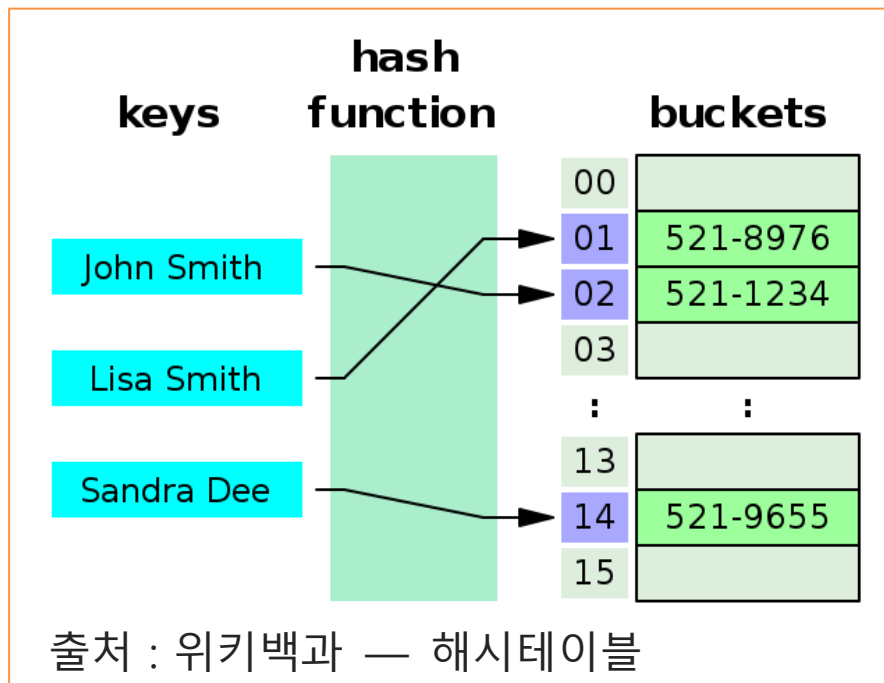
## ❖ 해싱의 정의

- 키 값에 직접 산술적인 연산을 적용하여 항목이 저장되어 있는 테이블의 주소를 계산하여 바로 찾아가는 계산 검색 방식이다.
- 검색방법
  - 키 값에 대해서 해시 함수를 계산하여 주소를 구하고,
  - 구한 주소에 해당하는 해시 테이블로 바로 이동
    - 해당 주소에 찾는 항목이 있으면 검색 성공, 없으면 검색 실패
- 해시 함수(hash function)
  - 키 값을 원소의 위치로 변환하는 함수

## ❖ 해싱의 정의

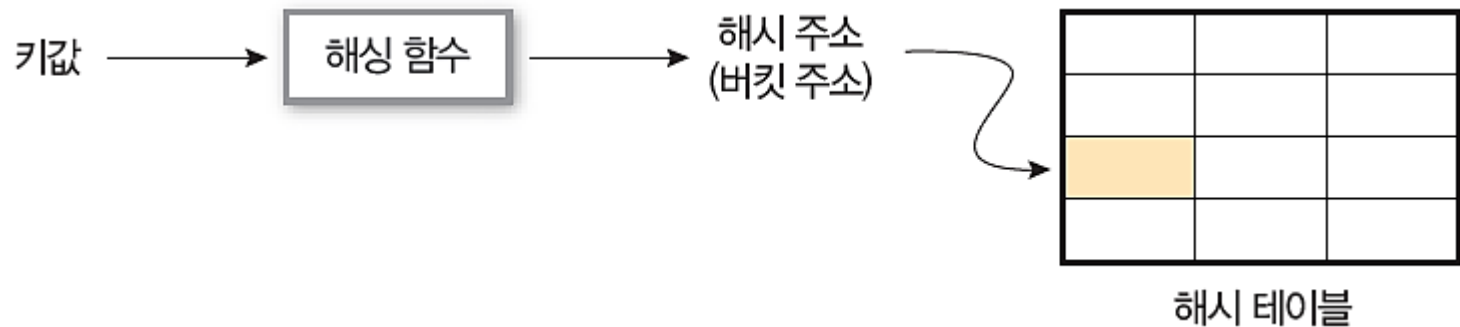
### ■ 해시 테이블

- 해시 함수에 의해 계산된 주소의 위치에 항목을 저장한 표
- 해시 테이블 구조(hash table structure)

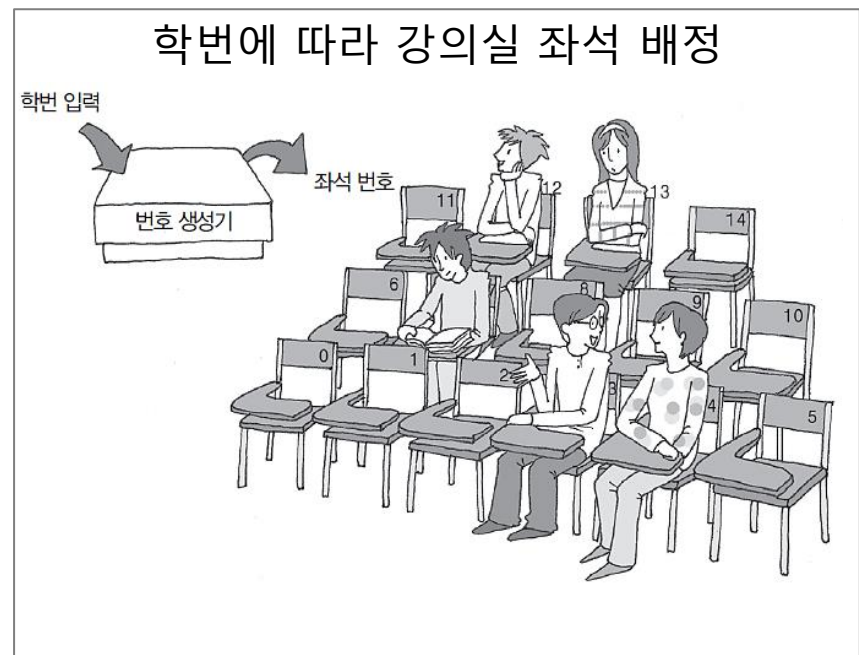
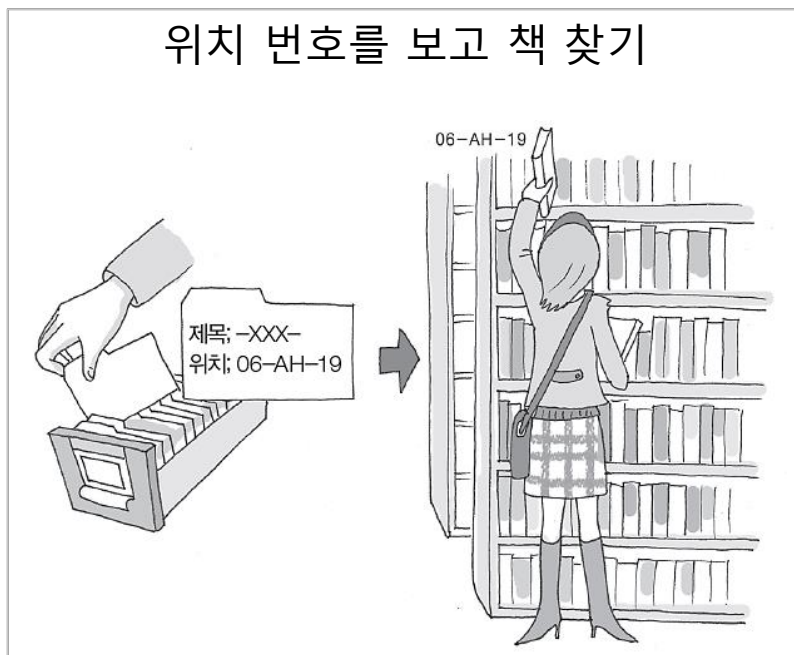


# 해싱(Hashing)

## ❖ 해싱 수행 방법



## ❖ 해싱의 예



## ❖ 해싱 관련 용어

- 동거자(synonym)
  - 서로 다른 키 값을 가지지만 해싱 함수에 의해 같은 버킷에 저장된 키 값들
- 충돌(collision)
  - 서로 다른 키 값에 대해서 해싱 함수에 의해 주어진 버킷 주소가 같은 경우
  - 충돌이 발생한 경우에 비어있는 슬롯에 동거자 관계로 키 값 저장
  - 오버플로 : 버킷에 비어있는 슬롯이 없는 포화 버킷 상태에서 충돌이 발생하여 해당 버킷에 키 값을 저장할 수 없는 상태



## ❖ 해싱 관련 용어

### ■ 키 값 밀도

- 사용 가능한 전체 키 값들 중에서 현재 해시 테이블에 저장되어서 실제 사용되고 있는 키 값의 개수 정도
- 키 값 밀도 = 실제 사용 중인 키 값의 개수 / 사용 가능한 키 값의 개수

### ■ 적재 밀도

- 해시 테이블에 저장 가능한 키 값의 개수 중에서 현재 해시 테이블에 저장되어서 실제 사용되고 있는 키 값의 개수 정도
- 적재 밀도 = 실제 사용 중인 키 값의 개수 /  
해시 테이블에 저장 가능한 전체 키 값의 개수  
= 실제 사용 중인 키 값의 개수 /  
(버킷 개수 × 슬롯 개수)

## ❖ 해시 함수의 조건

- 해시 함수는 계산이 쉬워야 함
  - 비교 검색 방법을 사용하여 키 값의 비교연산을 수행하는 시간보다 해싱 함수를 사용하여 계산하는 시간이 빨라야 해싱 검색을 사용하는 의미가 됨
- 해시 함수는 충돌이 적어야 함
  - 충돌이 많이 발생한다는 것은 같은 버킷을 할당 받는 키 값이 많다는 것이므로 비어있는 버킷이 많은데도 어떤 버킷은 오버플로가 발생할 수 있는 상태가 되므로 좋은 해시 함수가 될 수 없음
- 해시 테이블에 고르게 분포할 수 있도록 주소를 만들어야 함

## ❖ 중간 제곱 함수

- 키 값을 제공한 결과 값에서 중간에 있는 적당한 비트를 주소로 사용
- 제공한 값의 중간 비트들은 대개 키의 모든 값과 관련이 있기 때문에 서로 다른 키 값은 서로 다른 해싱 주소를 갖게 됨

(예) 키 값 00110101 10100111에 대한 해시 주소 구하기

X

|          |      |           |      |          |
|----------|------|-----------|------|----------|
| 00001011 | 0011 | 1110 1001 | 0010 | 11110001 |
|----------|------|-----------|------|----------|

해시주소

## ❖ 제산(Division) 함수

- 함수는 나머지 연산자 mod(C에서의 %연산자)를 사용하는 방법
- 키 값 k를 해시 테이블의 크기 M으로 나눈 나머지를 해시 주소로 사용
- M으로 나눈 나머지 값은 0~(M-1)이 되므로 해시 테이블의 인덱스로 사용
- 해시 주소는 충돌 발생 없이 고르게 분포하도록 생성되어야 하므로 키 값을 나누는 해시 테이블의 크기 M은 적당한 크기의 소수 (prime number) 사용

$$h(k) = k \bmod M$$

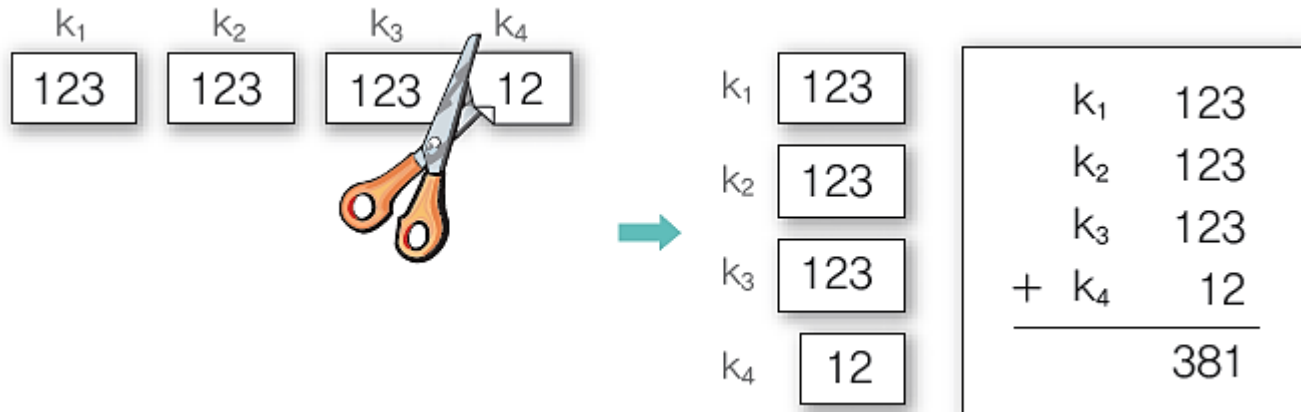
## ❖ 승산 함수

- 곱하기 연산을 사용하는 방법
- 키 값  $k$ 와 정해진 실수  $\alpha$ 를 곱한 결과에서 소수점 이하 부분만을 테이블의 크기  $M$ 과 곱하여 그 정수 값을 주소로 사용

$$h(k) = (k\alpha \bmod 1) * M$$

## ❖ 접지 함수

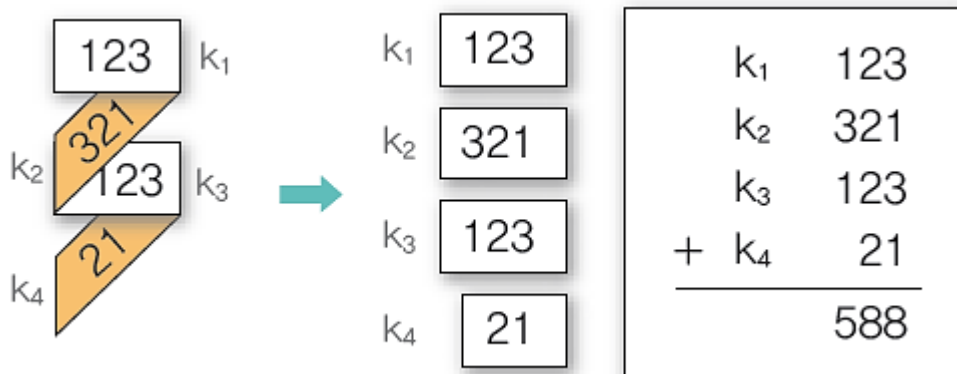
- 키의 비트 수가 해시 테이블 인덱스의 비트 수보다 큰 경우에 주로 사용
- 이동 접지 함수
  - 각 분할 부분을 이동시켜서 오른쪽 끝자리가 일치하도록 맞추고 더하는 방법
  - 예) 해시 테이블 인덱스가 3자리이고 키 값  $k$ 가 12312312312인 경우
  - 이동 접지 함수의 사용 예



## ❖ 접지 함수

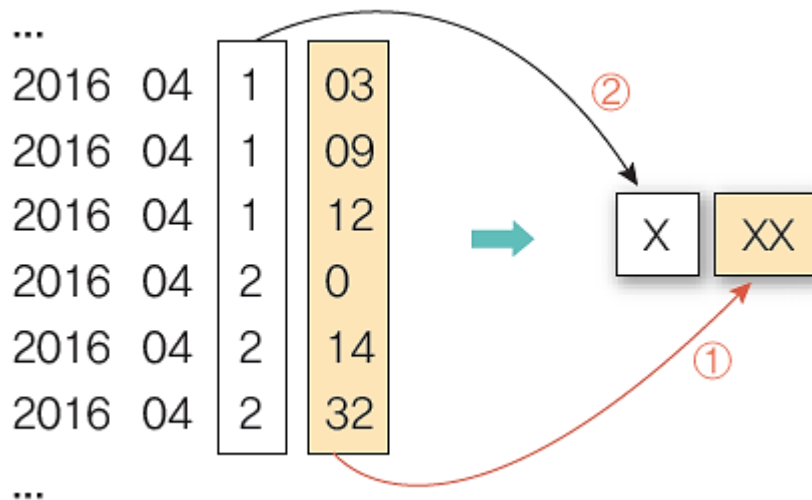
### ■ 경계 접지 함수

- 분할된 각 경계를 기준으로 접으면서 서로 마주보도록 배치하고 더하는 방법
- 예) 해시 테이블 인덱스가 3자리이고 키 값  $k$ 가 12312312312인 경우
- 경계 접지 함수의 사용 예



## ❖ 숫자 분석 함수

- 키 값을 이루고 있는 각 자릿수의 분포를 분석하여 해시 주소로 사용
- 각 키 값을 적절히 선택한 진수로 변환한 후에 각 자릿수의 분포를 분석하여 가장 편중된 분산을 가진 자릿수는 생략하고, 가장 고르게 분포된 자릿수부터 해시 테이블 주소의 자릿수만큼 차례로 뽑아서 만든 수를 역순으로 바꾸어서 주소로 사용
- 예) 키 값이 학번이고 해시 테이블 주소의 자릿수가 3자리인 경우 숫자분석 함수의 사용 방법





## ❖ 진법 변환 함수

- 키 값이 10진수가 아닌 다른 진수일 때, 10진수로 변환하고 해시 테이블 주소로 필요한 자릿수만큼만 하위자리의 수를 사용하는 방법

## ❖ 비트 추출 함수

- 해시 테이블의 크기가  $2^k$ 일 때 키 값을 이진 비트로 놓고 임의의 위치에 있는 비트들을 추출하여 주소로 사용하는 방법
- 이 방법에서는 충돌이 발생할 가능성이 많으므로 테이블의 일부에 주소가 편중되지 않도록 키 값들의 비트들을 미리 분석하여 사용해야 함

## ❖ 선형 개방 주소법 (선형 조사법 linear probing)

- 해싱 함수로 구한 버킷에 빈 슬롯이 없어서 오버플로가 발생하면, 그 다음 버킷에 빈 슬롯이 있는지 조사
  - 빈 슬롯이 있으면 - 키 값을 저장
  - 빈 슬롯이 없으면 - 다시 그 다음 버킷을 조사
  - 이런 과정을 되풀이 하면서 해시 테이블 내에 비어있는 슬롯을 순차적으로 찾아서 사용하여 오버플로 문제를 처리하는 방법

# 해싱에서 충돌(오버플로)를 처리하는 방법

## ❖ 선형 개방 주소법 (선형 조사법 linear probing)

- 선형 개방 주소법을 이용한 오버플로 처리 예
  - 해시 테이블 크기 : 5
  - 해시 함수 : 제산 함수 사용. 해시 함수  $h(k) = k \bmod 5$
  - 저장할 키 값 : {45, 9, 10, 96, 25}

①  $h(45) = 45 \bmod 5 = 0 \rightarrow$  해시 테이블 0번에 키 값 45 저장

|   |    |
|---|----|
| 0 | 45 |
| 1 |    |
| 2 |    |
| 3 |    |
| 4 |    |

# 해싱에서 충돌(오버플로)를 처리하는 방법

## ❖ 선형 개방 주소법 (선형 조사법 linear probing)

②  $h(9) = 9 \bmod 5 = 4 \rightarrow$  해시 테이블 4번에 키 값 9 저장

|   |    |
|---|----|
| 0 | 45 |
| 1 |    |
| 2 |    |
| 3 |    |
| 4 | 9  |

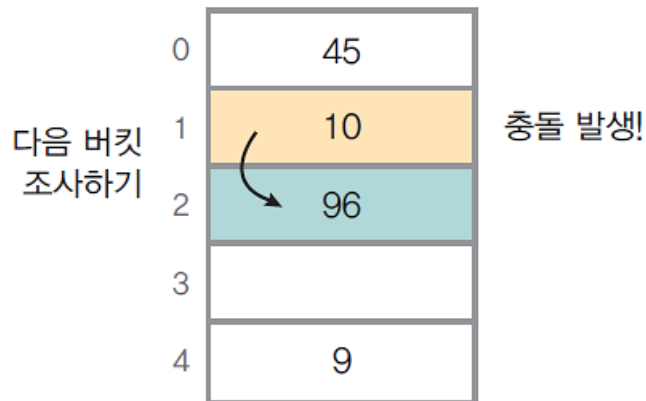
③  $h(10) = 10 \bmod 5 = 0 \rightarrow$  충돌 발생 !  $\rightarrow$  다음 버킷 중에서 비어 있는 버킷 1에 키 값 10을 저장

|               |   |    |        |
|---------------|---|----|--------|
| 다음 버킷<br>조사하기 | 0 | 45 | 충돌 발생! |
|               | 1 | 10 |        |
|               | 2 |    |        |
|               | 3 |    |        |
|               | 4 | 9  |        |

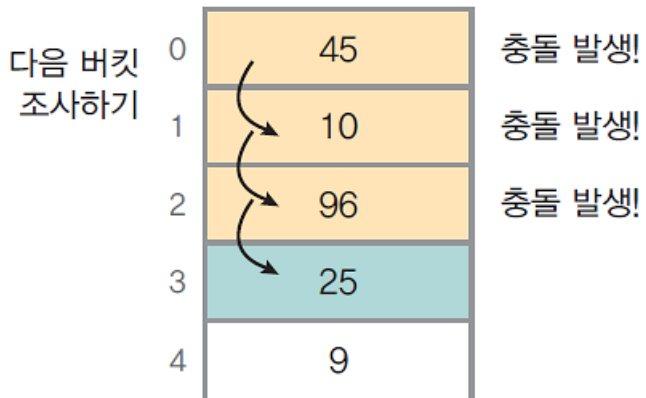
# 해싱에서 충돌(오버플로)를 처리하는 방법

## ❖ 선형 개방 주소법 (선형 조사법 linear probing)

- ④  $h(96) = 96 \bmod 5 = 1 \rightarrow$  충돌 발생!  $\rightarrow$  다음 버킷 중에서 비어 있는 버킷 2에 키 값 96을 저장



- ⑤  $h(25) = 25 \bmod 5 = 0 \rightarrow$  충돌 발생!  $\rightarrow$  다음 버킷 중에서 비어 있는 버킷 3에 키 값 25를 저장



# 해싱에서 충돌(오버플로)를 처리하는 방법

## ❖ 선형 조사법 프로그램

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define KEY_SIZE 12      // 탐색키의 최대길이
#define TABLE_SIZE 13  // 해싱 테이블의 크기=소수

typedef struct
{
    char key[KEY_SIZE];
    // 다른 필요한 필드들
} element;

element hash_table[TABLE_SIZE];    // 해싱 테이블
```

## ❖ 선형 조사법 프로그램

```
// 문자로 된 키를 숫자로 변환
int transform1(char *key)
{
    int number = 0;
    while (*key)
        number = number + *key++;
    return number;
}
// 제산 함수를 사용한 해싱 함수
int hash_function(char *key)
{
    // 키를 자연수로 변환한 다음 테이블의 크기로 나누어 나머지를 반환
    return transform1(key) % TABLE_SIZE;
}

#define empty(item) (strlen(item.key)==0)
#define equal(item1, item2) (!strcmp(item1.key,item2.key))
```

## ❖ 선형 조사법 프로그램

```
// 선형 조사법을 이용하여 테이블에 키를 삽입하고,  
// 테이블이 가득 찬 경우는 종료  
void hash_lp_add(element item, element ht[])  
{  
    int i, hash_value;  
    hash_value = i = hash_function(item.key);  
    //printf("hash_address=%d\n", i);  
    while (!empty(ht[i])) {  
        if (equal(item, ht[i])) {  
            fprintf(stderr, "탐색키가 중복되었습니다\n");  
            exit(1);  
        }  
        i = (i + 1) % TABLE_SIZE;  
        if (i == hash_value) {  
            fprintf(stderr, "테이블이 가득 찼습니다\n");  
            exit(1);  
        }  
    }  
    ht[i] = item;  
}
```



## ❖ 선형 조사법 프로그램

```
// 선형조사법을 이용하여 테이블에 저장된 키를 탐색
void hash_lp_search(element item, element ht[])
{
    int i, hash_value;
    hash_value = i = hash_function(item.key);
    while (!empty(ht[i]))
    {
        if (equal(item, ht[i])) {
            fprintf(stderr, "탐색 %s: 위치 = %d\n", item.key, i);
            return;
        }
        i = (i + 1) % TABLE_SIZE;
        if (i == hash_value) {
            fprintf(stderr, "찾는 값이 테이블에 없음\n");
            return;
        }
    }
    fprintf(stderr, "찾는 값이 테이블에 없음\n");
}
```

## ❖ 선형 조사법 프로그램

```
// 해싱 테이블의 내용을 출력
void hash_lp_print(element ht[])
{
    int i;
    printf("\n===== \n");
    for (i = 0; i < TABLE_SIZE; i++)
        printf("[%d] %s\n", i, ht[i].key);
    printf("===== \n");
}
```

## ❖ 선형 조사법 프로그램

```
int main(void)
{
    int i;
    char *s[7] = { "do", "for", "if", "case", "else", "return", "function" };
    element e;

    for (i = 0; i < 7; i++) {
        strcpy(e.key, s[i]);
        hash_lp_add(e, hash_table);
        hash_lp_print(hash_table);
    }
    for (i = 0; i < 7; i++) {
        strcpy(e.key, s[i]);
        hash_lp_search(e, hash_table);
    }
    return 0;
}
```

# 해싱에서 충돌(오버플로)를 처리하는 방법

## ❖ 선형 조사법 프로그램

- 실행 결과 : { "do", "for", "if", "case", "else", "return", "function" }

```
C:\WINDOWS\system32\cmd.exe
=====
[0]
[1] do
[2]
[3]
[4]
[5]
[6]
=====

[0]
[1] do
[2]
[3]
[4]
[5] for
[6]
=====

[0]
[1] do
[2]
[3]
[4] if
[5] for
[6]
=====
```

```
C:\WINDOWS\system32\cmd.exe
=====
[0]
[1] do
[2]
[3]
[4] if
[5] for
[6] case
=====

[0] else
[1] do
[2]
[3]
[4] if
[5] for
[6] case
=====

[0] else
[1] do
[2] return
[3]
[4] if
[5] for
[6] case
=====
```

```
C:\WINDOWS\system32\cmd.exe
=====
[0] else
[1] do
[2] return
[3] function
[4] if
[5] for
[6] case
=====
탐색 do: 위치 = 1
탐색 for: 위치 = 5
탐색 if: 위치 = 4
탐색 case: 위치 = 6
탐색 else: 위치 = 0
탐색 return: 위치 = 2
탐색 function: 위치 = 3
계속하려면 아무 키나 누르십시오 . . .
```

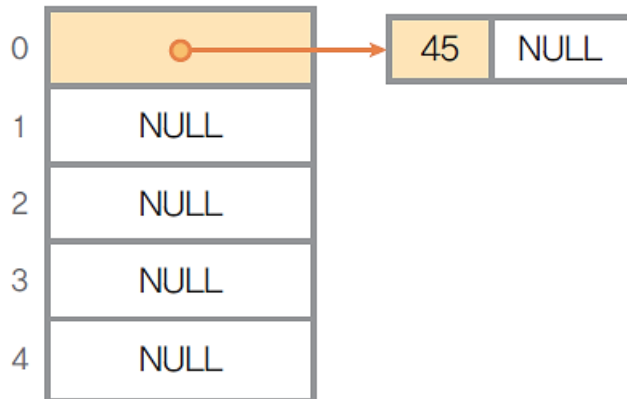
## ❖ 체이닝

- 해시 테이블 구조를 변경하여 각 버킷에 하나 이상의 키 값을 저장 할 수 있도록 하는 방법
- 버킷에 슬롯을 동적으로 삽입하고 삭제하기 위해서 연결 리스트 사용
  - 각 버킷에 대한 헤드 노드를 1차원 배열로 만들고 각 버킷에 대한 헤드 노드는 슬롯들을 연결 리스트로 가지고 있어서 슬롯 삽입이나 삭제 연산을 쉽게 수행 가능
  - 버킷 내에서 원하는 슬롯을 검색하기 위해서는 버킷의 연결 리스트를 선형 검색

## ❖ 체이닝

- 체이닝을 이용한 오버플로 처리 예
  - 해시 테이블의 크기 : 5
  - 해시 함수 : 제산함수 사용. 해시 함수  $h(k) = k \bmod 5$
  - 저장할 키 값 : {45, 9 10, 96, 25}

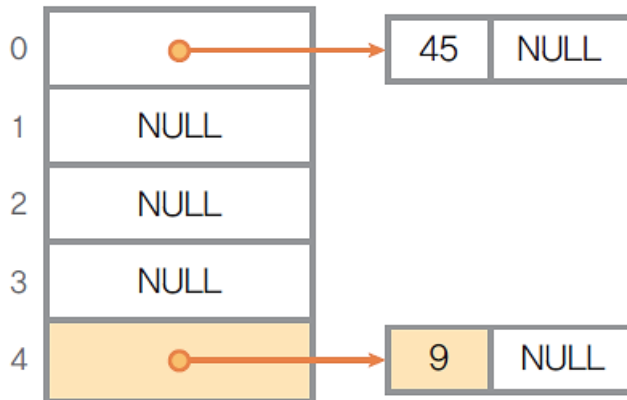
①  $h(45) = 45 \bmod 5 = 0 \rightarrow$  해시 테이블 0번에 키 값 45를 저장



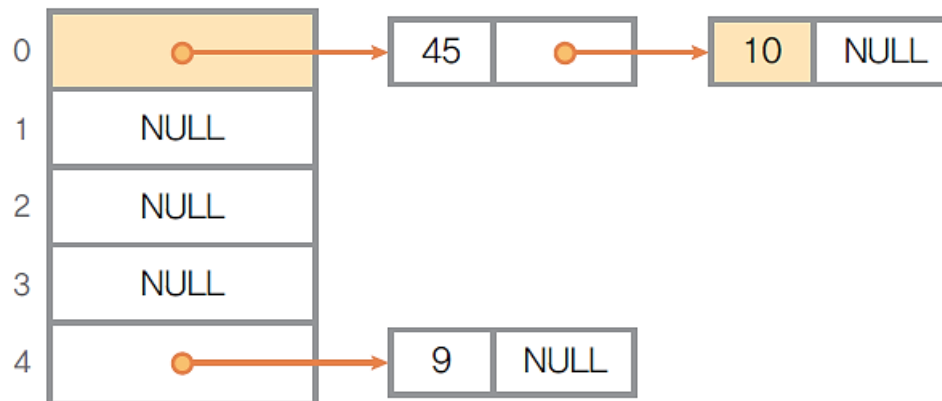
# 해싱에서 충돌(오버플로)를 처리하는 방법

## ❖ 체이닝

②  $h(9) = 9 \bmod 5 = 4 \rightarrow$  해시 테이블 4번에 노드를 삽입  
키 값 9를 저장



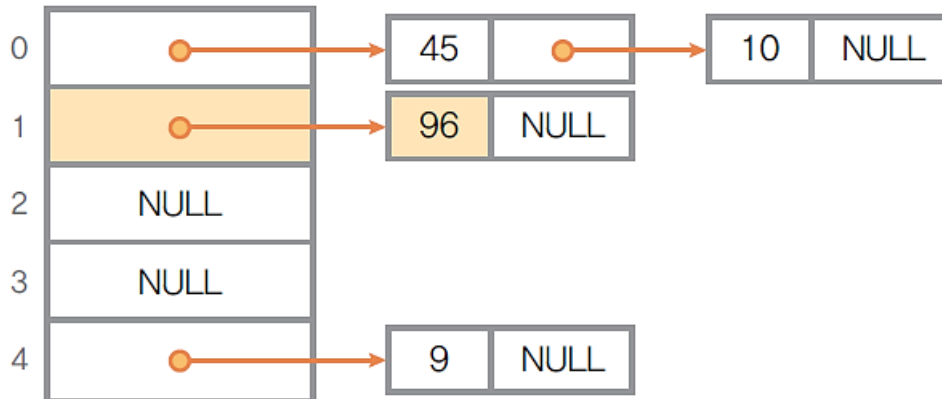
③  $h(10) = 10 \bmod 5 = 0 \rightarrow$  해시 테이블 0번에 노드를 삽입  
키 값 10을 저장



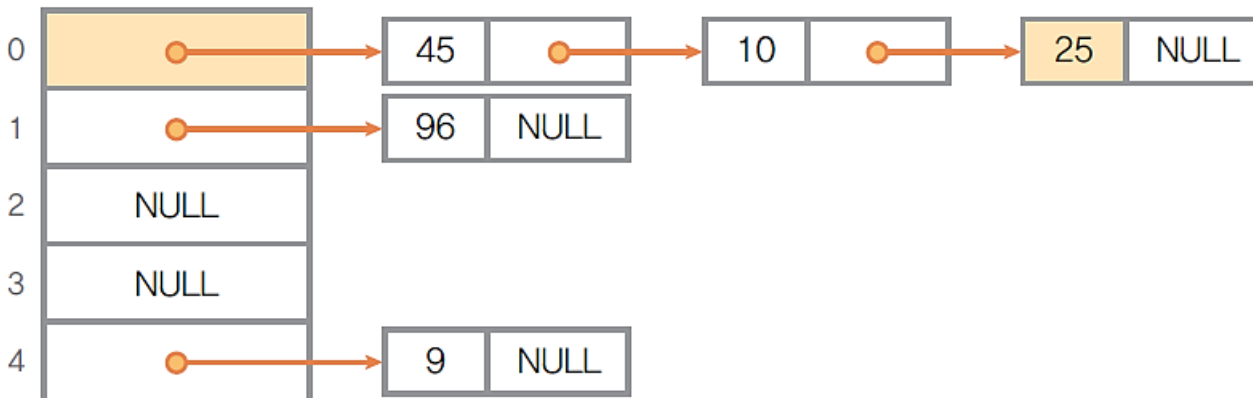
# 해싱에서 충돌(오버플로)를 처리하는 방법

## ❖ 체이닝

- ④  $h(96) = 96 \bmod 5 = 1 \rightarrow$  해시 테이블 1번에 노드를 삽입  
키 값 96을 저장



- ⑤  $h(25) = 25 \bmod 5 = 0 \rightarrow$  해시 테이블 0번에 노드를 삽입 키 값 25를 저장





## ❖ 체이닝 프로그램

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define TABLE_SIZE5 // 해싱 테이블의 크기=소수

typedef struct {
    int key;
} element;
struct list
{
    element item;
    struct list *link;
};
struct list *hash_table[TABLE_SIZE];

// 제산 함수를 사용한 해싱 함수
int hash_function(int key)
{
    return key % TABLE_SIZE;
}
```

## ❖ 체이닝 프로그램

```
// 체인법을 이용하여 테이블에 키를 삽입
void hash_chain_add(element item, struct list *ht[])
{
    int hash_value = hash_function(item.key);
    struct list *ptr;
    struct list *node_before = NULL, *node = ht[hash_value];
    for (; node; node_before = node, node = node->link) {
        if (node->item.key == item.key) {
            fprintf(stderr, "이미 탐색키가 저장되어 있음\n");
            return;
        }
    }
    ptr = (struct list *)malloc(sizeof(struct list));
    ptr->item = item;
    ptr->link = NULL;
    if (node_before)
        node_before->link = ptr;
    else
        ht[hash_value] = ptr;
}
```

## ❖ 체이닝 프로그램

```
// 체인법을 이용하여 테이블에 저장된 키를 탐색
void hash_chain_search(element item, struct list *ht[])
{
    struct list *node;

    int hash_value = hash_function(item.key);
    for (node = ht[hash_value]; node; node = node->link) {
        if (node->item.key == item.key) {
            fprintf(stderr, "탐색 %d 성공 \n", item.key);
            return;
        }
    }
    printf("키를 찾지못했음\n");
}
```

## ❖ 체이닝 프로그램

```
// 해싱 테이블의 내용을 출력
void hash_chain_print(struct list *ht[])
{
    struct list *node;
    int i;
    printf("\n===== \n");
    for (i = 0; i < TABLE_SIZE; i++) {
        printf("[%d]-> ", i);
        for (node = ht[i]; node; node = node->link) {
            printf("%d-> ", node->item.key);
        }
        printf("\n");
    }
    printf("===== \n");
}
```

## ❖ 체이닝 프로그램

```
// 해싱 테이블을 사용한 예제
int main()
{
    int i;
    int data[TABLE_SIZE] = { 45, 9, 10, 96, 25 };
    element e;

    for (i = 0; i < TABLE_SIZE; i++) {
        e.key = data[i];
        hash_chain_add(e, hash_table);
        hash_chain_print(hash_table);
    }
    for (i = 0; i < TABLE_SIZE; i++) {
        e.key = data[i];
        hash_chain_search(e, hash_table);
    }
    return 0;
}
```

# 해싱에서 충돌(오버플로)를 처리하는 방법

## ❖ 체이닝 프로그램

### ■ 실행 결과

```
C:\WINDOWS\system32\cmd.exe

=====
[0] ->45->
[1] ->
[2] ->
[3] ->
[4] ->
=====

=====
[0] ->45->
[1] ->
[2] ->
[3] ->
[4] ->9->
=====

=====
[0] ->45->10->
[1] ->
[2] ->
[3] ->
[4] ->9->
=====
```

```
C:\WINDOWS\system32\cmd.exe

=====
[0] ->45->10->
[1] ->96->
[2] ->
[3] ->
[4] ->9->
=====

=====
[0] ->45->10->25->
[1] ->96->
[2] ->
[3] ->
[4] ->9->
=====

탐색 45 성공
탐색 9 성공
탐색 10 성공
탐색 96 성공
탐색 25 성공
계속하려면 아무 키나 누르십시오
. . .
```